Build an MCP server (Model Context Protocol, stdio transport) that an LLM running inside OpenCode uses to answer questions about two data sources behind an OAuth-protected API

The assessment is split into three subtasks:

1. Authentication OAuth 2.1 client_credentials handshake.
2. BOM query a per-student bill of materials (CSV)
3. 10-K-extract structured information from Alphabet's FY2024 10-K filling (POF).

You may implement the server in any language with an MCP SDK.

Provided template & models

The submission template you download already contains a .env file with an **OpenRouter API key**. Use it to run your agent (eg. inside **OpenCode**) no separate account, sign-up, or billing is required. The key is restricted to the following models:

- moonshotai/kimi-k2.5
- minimax/minimax-2.5

Configure OpenCode to use one of these models with the key from env

Connection details

Setting	**Value**
API_BASE_URL	http://assessment.seal.cit.tum.de
Access	EDU VPN required. Install and connect via eduVPN before any request.
CLIENT_ID	Your 8-digit Matrikelnummer (on your handout)
CLIENT_SECRET	On your handout
OAuth grant	client_credentials
Token TTL	300 seconds

Discovery endpoints:	/openapi.json, /.well-known/oauth-authorization-server, /.well-known/oauth-protected-resource

Subtask 1- Authentication

MCP tool to build

Tool	**Arguments**	**Returns**
authenticate	client id: str, client secret: str	(authenticated: True, client_id, issuer, expires_in_seconds) an success, (authenticated: False, error: ...) on failure

Credentials are passed as tool arguments at runtime (not via environment). On success, cache them in-process so subsequent data tools can refresh the OAuth token. Any data tool called before a successful authenticate must return a clear error.

Prompt to test

Authenticate me. My CLIENT_ID is <your-id and my CLIENT_SECRET is <your-secret>.

Expected result

The agent calls authenticate(...) and reports back something like ("authenticated": true, "client_id": " <your-id>", "issuer": "http://assessment.seal.cit.tum.de", "expires_in_seconds": 300).

Subtask 2-BOM (per-student CSV)

/files/bom.csv returns a 20-row CSV unique to your CLIENT ID. Columns: product_id, product_name, quantity, unit, supplier

MCP tools to build

Read/query

Tool	Arguments	Returns
get_item	product id: str	matching row, or (error, similar, available_ ids) if not found. Case-insensitive on product_id

search_items	name_pattern: str	rows whose product_name contains name_pattern (case-insensitive substring)
list_items	supplier?, unit?, min_quantity?, max_quantity?	filtered rows; no filters returns the full BOM
aggregate	group by: str, metric: str	grouped totals. group_by (supplier, unit); metric e (count, sum quantity). Sorted by metric descending.

Domain analysis

Tool	Arguments	Returns
supplier_breakdown	-	

Prompt to test

Analyse my BOM list every item, aggregate totals supplier, and tell me whether supplier dominates

Expected result

The agent calls List items() (or get items for spot checks), aggregate (group by supplier, metric="sum quantity"), and supplier_breakdown(). It reports

- the 20-row BOM,
- the supplier with the highest summed quantity,
- the dominant supplier's share_percent and the overall hhi, with a one-sentence interpretation.

Subtask 3-10-K (shared Alphabet FY2024 filing)

/files/10k.pdf returns Alphabet's FY2024 10-K filing. Same file for every student

MCP tools to build

Tool	**Arguments**	**Returns**

list_sections	-	TOC entries: [{title, page, level, item id}, …] . item_id is 1A, 7, NOTE-10, … or null for non-item/Note entries.
get_section	item_id: str	{title, item_id, start_page, end_page, text}. Must accept "Item 1A", "1A", "Note 10", or a partial title.
get_financial_statement	name: str, years int?	{title, page, columns, rows} where each row is {line_item, <year1>, <year2>,...} name {balance_sheet, income, cash_flow, comprehensive_income, stockholders_equity). Optional year filters columns.
search	query: str, scope: str?	[{text, page, bbox}, …]. bbox is [x0, top, x1, botton] in PDF points. Optional scope is an item_id that limits the search.
resolve_reference	ref: str	Same shape as get_section. Accepts "Note 10", "Item 1A, etc.

Prompt to test

Summarise Item 1A in three bullets, then give me Alphabet's revenue history from the income statement, and finally tell me what Note 10 covers-cite the page numbers.

Expected result

The agent calls get_section("Item 1A") (or search for citations), get financial_statement("incone"), and resolve_reference("Note 10"). It reports:

- three bullet-point risks from Item 1A with page citations,
- the Revenues row from the income statement: 30,394/350,018/402,836 for 2023/2024/2025,
- Note 10's title (Commitments and Contingencies) and page range.

Submission

Upload one zip named <matrikelnummer>.zip containing:

server/	#your MCP server source code
opencode.jsonc	#OpenCode config (without CLIENT SECRET)
answers.ipynb	#answers to the test prompts above
opencode_answer_log.nd	#the OpenCode session transcript
agent log. (md, json, txt)	#log of the agent/tool you used to solve the tasks
README.md	#how to run your server (5-10 Lines)

agent log.* may be one or multiple files, in any of the listed formats.

Do not include CLIENT SECRET in any submitted file.